

STATE MIND

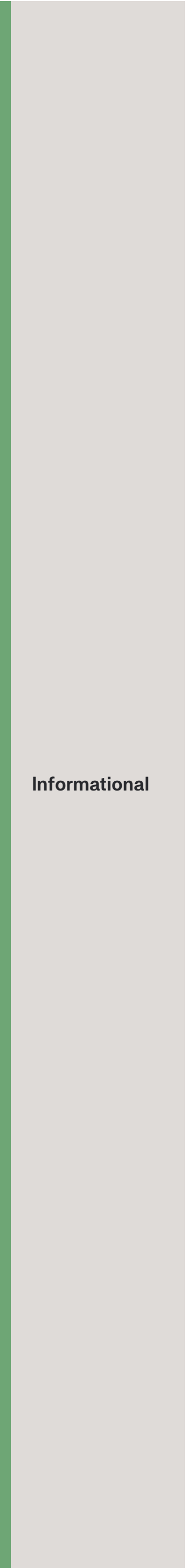
Symbiotic Core V2

12-02-2026 – 20-04-2026

Table of contents



1. Project brief		3
2. Finding severity breakdown		5
3. Summary of findings		6
4. Conclusion		6
5. Findings report		7
High	Withdrawal requests can be repriced at the unlock boundary during new bucket creation	7
	Shared UniversalDelegator.setSize() can reduce or cancel an already requested slash	8
	Plugin collateral is not deallocated when vault liquidity is required	9
Medium	User funds are not slashable and not claimable when block.timestamp == unlockAfter	10
	Stale withdrawal buffer prevSum after removing the last root group	10
	Pending accounting underreports when clear events outlive their paired add events in the rolling window	11
	If an operator in a shared group is slashed, every operator in the group is affected	12
	Non-Zero size persists after slot removal	12
	UniversalDelegator.setSize() increase during pending window inflates	13
	UniversalDelegator.getAllocated() beyond original allocation	13
	UniversalDelegator.removeSlot() and UniversalDelegator.resetAllocation() unexpected behavior	14
UniversalDelegator.removeSlot() does not decrement _noPluginsSize for noPlugins group slots	17	
Legacy slash execution may keep pending allocation slashable and overstate slot stake	18	
Informational	Redundant VaultV2.deallocatePlugins() calls in claimBatch() function	19



Informational

Missing custom error for non-existent plugins in VaultV2.swapPlugins()	19
Incorrect storage gap update in the VaultV2Storage contract	20
Wrong parameter in VaultV2.instantWithdraw() event emission	20
Unused storage variable in UniversalDelegator	21
Unused _rootSlot function in UniversalDelegator	21
Claimers cannot determine which withdrawal index to claim, enabling griefing and breaking claim flow	22
Confusing naming for withdrawal count and withdraw/redeem return value	23
VaultV2.donate() distributes rewards proportionally but can be gamed via sandwich attack	23
VaultV2 interface contains unused errors and omits declared IVaultV2.ClaimBatch emission	24
Gas optimization	24
VaultV2 slashing can be blocked by plugin deallocation revert	25
VaultV2 trusts plugin deallocate return value and can underflow accounting	25
UniversalDelegator.resetAllocation() may emit a parent index instead of the subnetwork slot index	26
Gas optimization 2	26
Post-migration slashing accepts boundary capture timestamps	27
UniversalDelegator.createSlot() does not validate subnetwork against network registry	27
PluginRegistry lacks de-whitelist functionality allowing permanently whitelisted compromised plugins	28
Legacy request UniversalSlasher.vetoSlash() writes completed flag to new slasher storage but not old slasher, creating inconsistent cross-contract state	28
Missing check in UniversalSlasher.setResolver()	29
Legacy slash request resolver lookup doesn't match VetoSlasher	30
Post-migration UniversalSlasher.slashableStake() ignores captureTimestamp semantics	31
Incomplete V2 interfaces force concrete casts in UniversalSlasher	32
Gas optimization 3	32

1. Project brief



Title	Description
Client	Symbiotic
Project name	Symbiotic Core V2
Timeline	12-02-2026 – 20-04-2026

Project Log

Date	Commit Hash	Note
13-02-2026	6ed667b4ce82dd712b13145653c5e2b4a2e4cccb	Initial Commit
20-02-2026	53b68070799a9d456856cde00723b4cc0d0bc80e	Updated commit
11-03-2026	05c3b867bbf1203c68b5cddd9d19716e4ab0ae8d	Reaudit commit with fixes
30-03-2026	90337df6119cb8e09d7f2eebb585d9028197c7db	Reaudit commit with UniversalDelegator refactor
13-04-2026	6682d8b7c407a60c47ff52dcfe30ae36d5f93e41	Reaudit commit 2
20-04-2026	00c916f29bb789312f6b856a964e2258a171b931	Reaudit commit 3
20-04-2026	05b78cb35bdb165763d6c731829ba82934ab5994	Reaudit commit 4

Short Overview

Symbiotic Core V2 is an updated version of the Symbiotic core protocol that simplifies the system architecture and changes how vault liquidity, delegation, and slashing are managed. It replaces the multi-option design used in Core V1 with a single configurable component for each core part of the system.

New features include:

- Vault V2
 - Introduction of adapters for fund's allocation from the Vault, allowing curators to set absolute limits for Symbiotic whitelisted integrations and move idle liquidity.
 - Support for instant withdrawals.
 - Support for donations.
- UniversalDelegator
 - Refactored allocation management logic.
 - Allocation management is now based on a slot mechanism.

- Each slot defines its own properties, including allocation size and children with their priorities.
- UniversalSlasher
 - Supports both the simple slasher and the veto slasher options.

Project Scope

The audit covered the following files:

 [VaultV2.sol](#)

 [VaultV2Storage.sol](#)

 [UniversalDelegator.sol](#)

 [UniversalDelegatorIndex.sol](#)

 [UniversalSlasher.sol](#)

 [PluginRegistry.sol](#)

2. Finding severity breakdown



All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Bugs leading to assets theft, fund access locking, or any other loss of funds to be transferred to any party.
High	Bugs that can trigger a contract failure. Further recovery is possible only by manual modification of the contract state or replacement.
Medium	Bugs that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss of funds.
Informational	Bugs that do not have a significant immediate impact and could be easily fixed.

Based on the feedback received from the Client regarding the list of findings discovered by the Contractor, they are assigned the following statuses:

Status	Description
Fixed	Recommended fixes have been made to the project code and no longer affect its security.
Acknowledged	The Client is aware of the finding. Recommendations for the finding are planned to be resolved in the future.

3. Summary of findings

Severity	# of Findings
Critical	0 (0 fixed, 0 acknowledged)
High	3 (3 fixed, 0 acknowledged)
Medium	9 (9 fixed, 0 acknowledged)
Informational	24 (15 fixed, 9 acknowledged)
Total	36 (27 fixed, 9 acknowledged)

4. Conclusion

During the audit of the codebase, 36 issues were found in total:

- 3 high severity issues (3 fixed)
- 9 medium severity issues (9 fixed)
- 24 informational severity issues (15 fixed, 9 acknowledged)

The final reviewed commit is 05b78cb35bdb165763d6c731829ba82934ab5994

5. Findings report



HIGH-01

Withdrawal requests can be repriced at the unlock boundary during new bucket creation

Fixed at:

66c11c9

Description

Lines:

- [VaultV2.sol#L208](#)

VaultV2.withdrawalsOf() resolves a request using

_unlockToBucket.upperLookupRecent(withdrawalUnlockAfter(index, account)), while **VaultV2.claim()** enforces **block.timestamp <= withdrawalUnlockAfter(...)** as not matured. At **block.timestamp == unlockAfter**, the request is still not claimable, but bucket state can still change at that same timestamp. This can happen in **VaultV2.onSlash()**, and also in **VaultV2.donate()** when it calls **_updateWithdrawalsSharePrice()** and pushes **_unlockToBucket**. A pre-existing request can therefore be remapped into a newer bucket at the boundary and repriced.

Actual state transition:

```
// request creation in _withdraw(), t0 - initial timestamp
_withdrawalUnlockAfter[0][claimer] = t0;

// later at boundary time (t1 == t0 + epochDuration), donate()/onSlash() -> _updateWithdrawalsSharePrice()
_unlockToBucket.push(t1, 1);
```

The request lifecycle is expected to be a half-open interval: active on **[t0, t1)** and claimable from **t1** onward. Under this model, resolving valuation with **upperLookupRecent(t1)** can cross into a post-boundary bucket and violate intended pre-maturity bucket assignment.

At **t1**, **_unlockToBucket.upperLookupRecent(t1)** already returns bucket **1**, so withdrawal request index **0** is remapped from bucket **0** to bucket **1** at the boundary. **VaultV2.claim()** is still blocked at **t1 (block.timestamp <= unlockAfter)**, and by the first claimable moment (**t1 + 1**) valuation is already taken from bucket **1**. In practice, the user cannot claim bucket **0** value for that request.

Impact: Withdrawal valuation may drift from intended bucket semantics at the unlock boundary. A claimant may receive more than expected after later donations, which may shift value from other participants or future inflows. The same state may also produce temporary claim failures.

POC: **VaultV2BoundarySlashPoCAliceBobTest.test_poc_boundarySlash_thenDonation_inflatesAliceWithdrawalValue()**

Recommendation

We recommend aligning request valuation so a request cannot be remapped at **unlockAfter**.

```
- uint256 bucketIndex = _unlockToBucket.upperLookupRecent(withdrawalUnlockAfter(index, account));
+ uint48 unlockAfter_ = withdrawalUnlockAfter(index, account);
+ uint48 lookupTimestamp = unlockAfter_ > 0 ? unlockAfter_ - 1 : 0;
+ uint256 bucketIndex = _unlockToBucket.upperLookupRecent(lookupTimestamp);
}
```


Description

Lines: [UniversalDelegator.sol#L596-L608](#)

`UniversalDelegator.setSize()` adds pending protection on size decreases only when `!parent.isShared`. For shared-parent slots, the pending branch is skipped, so the size is applied immediately. Since `UniversalDelegator.getAllocated()` uses the current size, the slashable exposure can drop before execution.

`UniversalSlasher.requestSlash()` records the `amount` as `min(amount, current slashableStake)` at request time, while `UniversalSlasher.executeSlash()` computes `min(request.amount, slashableStake(..., request.createdAt, ...))`. In the post-migration branch, `UniversalSlasher.slashableStake()` still returns the current `stakeFor(...)`, not a frozen snapshot at request creation. As a result, an already requested slash may be reduced or fully canceled if a curator decreases the shared-slot size during the veto/execution window.

Impact: slash behavior for shared slots becomes unstable across the request-to-execution interval. A slash request that is valid when submitted may later execute for a smaller amount or revert with `InsufficientSlash` due to an intermediate shared `setSize()` decrease.

Recommendation

We recommend aligning execution-time slashability with request-time exposure for shared slots. One option is to extend pending protection to shared-parent decreases so that exposure remains slashable for the intended window.

Description

Lines:

- [VaultV2.sol#L708](#)
- [VaultV2.sol#L724](#)

VaultV2 only deallocates plugin-held collateral through **VaultV2.deallocatePlugins()**, and that function pulls funds back only when **pluginsAllocated > totalStake()**. This condition is too narrow for the places where the vault actually needs liquidity.

The vault can require liquid collateral for slash settlement or user withdrawals even while **pluginsAllocated <= totalStake()**, because **totalStake()** is an accounting value, not a measure of how much collateral is immediately available in the vault. As a result, plugin-held collateral is not deallocated in several relevant paths, and operations that require liquidity either consume unrelated liquid funds first or revert even though the assets still exist inside plugins.

The first case is plugins-group slashing. **VaultV2.onSlash()** calls **VaultV2.deallocatePlugins()** before reducing **activeStake()**. At that point, **pluginsAllocated** is compared against the pre-slash **totalStake()**, so a plugins-group slash usually does not trigger any deallocation because the allocation is still within the current total stake. **VaultV2._availableToSlash()** then burns the vault's liquid balance, which may correspond to the no-plugins group rather than to the slashed plugins group. After the burn, **activeStake()** is reduced, but the plugin funds remain allocated. In the common case where **pluginsAllocated == totalStake()** after the slash, later calls to **VaultV2.deallocatePlugins()** still compute **toDeallocate = 0**, so the plugin collateral is never pulled back automatically.

The second case is owed-slash settlement. **VaultV2.syncOwedSlash()** does not call **VaultV2.deallocatePlugins()** at all and relies entirely on **VaultV2._availableToSlash()**, which only considers liquid collateral already sitting in the vault, which may be funds of a no plugins group.

The third case is matured withdrawal claims. Once the withdrawal matures, **VaultV2.claim()** and **VaultV2.claimBatch()** call **VaultV2.deallocatePlugins()**, but recovery still depends on the same **pluginsAllocated > totalStake()** heuristic. The vault may not receive enough funds and the final **safeTransfer()** in **claim()** reverts. In that state, users cannot claim matured withdrawals even though the collateral still exists inside the plugin.

All of these outcomes stem from the same root cause: deallocation is tied to an excess-allocation check instead of to the actual liquidity requirement of the operation being executed. The vault does not force plugin-backed collateral to return when liquidity is needed for a plugins-group slash, owed slash settlement, or matured claims.

Impact: The vault may burn liquid collateral that belongs to the no-plugins group to satisfy a plugins-group slash, which defeats the intended isolation of no-plugins funds from plugin risk. Slash settlement may remain partially unpaid while collateral is still stranded in plugins. Users with matured withdrawals may be unable to claim. In these cases, plugin funds may remain stuck until the Curator manually calls **VaultV2.deallocatePlugin()**, so the protocol does not autonomously recover liquidity when it is required.

Recommendation

We recommend

- Making deallocation depend on the liquidity required by the current operation rather than only on whether **pluginsAllocated > totalStake()**
- To fully settle slashes from the slashed group funds. In case of plugins group, vault should pull back plugin collateral. In case of no plugins group, it should use no-plugins liquidity.
- For **VaultV2.claim()** and **VaultV2.claimBatch()**, ensure that enough collateral is deallocated to satisfy the pending transfer.

MEDIUM-01	User funds are not slashable and not claimable when block.timestamp == unlockAfter	Fixed at: 59c75fa
-----------	--	--------------------------------------

Description

Lines:

- [VaultV2.sol#L381](#)
- [VaultV2.sol#L445](#)

When **block.timestamp == unlockAfter**, the withdrawal enters an inconsistent state. The user cannot claim the funds, and the funds are not slashable. This happens because **VaultV2.activeWithdrawals()** excludes the withdrawal from slashing, while **VaultV2.claim()** uses a **<=** check that still rejects claims at that exact timestamp. The withdrawal should be slashable or claimable. There should be no inconsistent state. Due to NatSpec in the [IVaultV2.sol](#), the withdrawal must be claimable.

Recommendation

We recommend changing the condition from **<=** to **<** so that the withdrawal becomes claimable:

```
if (block.timestamp < withdrawalUnlockAfter(index, msg.sender)) {
  revert WithdrawalNotMatured();
}
```

MEDIUM-02	Stale withdrawal buffer prevSum after removing the last root group	Fixed at: 0094cf0
-----------	---	--------------------------------------

Description

Line: [UniversalDelegator.sol#L719](#)

[UniversalDelegator.removeSlot\(\)](#) calls [UniversalDelegator._removeSlot\(\)](#) under the **UniversalDelegator.syncPrevSums()** modifier. When the removed slot is the first root child and its **nextSlot** is **WITHDRAWAL_BUFFER_CHILD_INDEX**, **UniversalDelegator._removeSlot()** sets **root.firstChild** to **0**. After the function returns, **UniversalDelegator.syncPrevSums()** calls [UniversalDelegator._syncPrevSums\(\)](#), but traversal starts from **firstChild**, so the withdrawal buffer slot is skipped, and its **prevSum** checkpoint is not refreshed. This leaves **slots[WITHDRAWAL_BUFFER_INDEX].prevSum** stale while **needPrevSumsSync** remains unset for that parent, so [UniversalDelegator._getPrevSum\(\)](#) returns the stale value directly. The same transition may also be reached through [UniversalDelegator.resetAllocation\(\)](#) when it collapses to the parent group (**existChildren == 1**) and then removes that last root group. Impact: [VaultV2.instantWithdraw\(\)](#) caps withdrawals using [UniversalDelegator.getWithdrawalBuffer\(\)](#). A stale withdrawal buffer **prevSum** can reduce instant withdraw capacity below intended availability until another root-child operation re-synchronizes the state.

Recommendation

We recommend updating [UniversalDelegator._removeSlot\(\)](#) to refresh the withdrawal buffer prefix state when removing the last root group, for example, by explicitly pushing **0** to **slots[WITHDRAWAL_BUFFER_INDEX].prevSum** in that branch.

Description

Lines:

- [UniversalDelegator.sol#L243-L250](#)
- [UniversalDelegator.sol#L262-L269](#)
- [UniversalDelegator.sol#L282-L289](#)
- [UniversalDelegator.sol#L301-L305](#)
- [UniversalDelegator.sol#L980-L987](#)

[UniversalDelegator.getChildrenPendingAt\(\)](#), [UniversalDelegator.getChildrenPending\(\)](#), [UniversalDelegator.getPendingAt\(\)](#), [UniversalDelegator.getPending\(\)](#), and [UniversalDelegator._getNoPluginsPending\(\)](#) estimate pending by looking only at the last **epochDuration** time window: pending increases in that window minus pending decreases in that window. The mismatch appears when a decrease is still inside that recent window, but the increase it originally canceled is already older than the window boundary. In that case, the decrease is still counted, the older increase is not, and the reported pending becomes lower than the real outstanding amount.

Example Scenario

1. **epochDuration = 3**, no-plugins group slot starts at size **200**.
2. At **t = 1**, **UniversalDelegator.setSize(group, 100)** creates **+100** pending. Reported pending is **100**.
3. At **t = 3**, **UniversalDelegator.onSlash(..., amount = 100)** clears **100** pending. Reported pending becomes **0**.
4. At **t = 4**, **UniversalDelegator.setSize(group, 50)** creates fresh **+50** pending.

At **t = 4**, these functions look only at the recent interval **(t - epochDuration, t] = (1, 4]**:

- Added pending in-window: **pendingCumulative.latest - pendingCumulative.upperLookupRecent(1) = 150 - 100 = 50**
- Cleared pending in-window:
clearedPendingCumulative.latest - clearedPendingCumulative.upperLookupRecent(1) = 100 - 0 = 100
- Reported pending: **saturatingSub(50, 100) = 0**

So the fresh **+50** pending at **t = 4** is masked by the older clear at **t = 3**, whose paired add (**t = 1**) fell out of the window boundary.

Impact: Underreporting from [UniversalDelegator.getPending\(\)](#) propagates into [UniversalDelegator.onSlash\(\)](#), where **pendingSlashed** is capped by reported pending before the remainder is applied to **size**, which may over-reduce slot size relative to intended pending clearance.

Underreporting from [UniversalDelegator.getChildrenPending\(\)](#) feeds into [UniversalDelegator.getAvailable\(\)](#), which may overstate parent availability.

Underreporting from [UniversalDelegator._getNoPluginsPending\(\)](#) feeds into [UniversalDelegator.getNoPluginsSize\(\)](#), which may understate reserved no-plugins capacity.

Recommendation

We recommend redesigning pending accounting so that outstanding exposure is tracked with explicit add/clear pairing across the slashable window, rather than independent cumulative window subtraction. As one practical implementation path, [UniversalDelegator.setSize\(\)](#) can apply epoch-delayed decreases by queuing reductions for one full **epochDuration** before finalization.

MEDIUM-04	If an operator in a shared group is slashed, every operator in the group is affected	Fixed at: 242f474
-----------	--	--------------------------------------

Description

Line: [UniversalDelegator.sol#L815](#)

In **UniversalDelegator.onSlash()**, the slash loop walks the full parent chain from the targeted operator slot up to the root, decrementing the size of every slot along the way.

When the targeted operator's slot is fully slashed, the full slash amount propagates up and removes the same quantity from the parent network slot and then from the grandparent group slot. For a shared group⁶ this is problematic because every sibling operator's effective allocation is derived from the group's size via **UniversalDelegator.getAllocated()**.

Consider the following concrete scenario. A shared group has size = 10. Two operators are registered under the same network inside that group, each with size = 10. Operator 1 is slashed for 10: its slot size becomes 0, the network slot size becomes 0, and the group slot size becomes 0. Operator 2's slot still records size = 10, but getAllocated for operator 2 now evaluates min(0, 10) = 0, so operator 2 reports zero stake even though it was never slashed.

Recommendation

We recommend separating the slash accounting at the group level so that reducing the slashed operator's contribution does not reduce the group's aggregate capacity for other operators.

MEDIUM-05	Non-Zero size persists after slot removal	Fixed at: 05c3b86
-----------	---	--------------------------------------

Description

Lines:

- [UniversalDelegator.sol#L687-L706](#)
- [UniversalDelegator.sol#L709-L735](#)

When a slot is removed, for example, a group slot, the slot is marked as non-existent; however, its size is not set to zero. This can cause phantom allocation in children of the group (operators and networks). For example, **UniversalDelegator.stakeFor(subnetwork, operator, 0)** will return a non-zero value, although the slot in which they exist no longer exists. Additionally, when a group is removed, mappings **_networkToSlot** and **_slotToNetwork** are not cleared. This can happen in the following sequence:

- activeStake = 200, group1.size = 100, group2.size = 100**
- group2** cannot be removed because **UniversalDelegator.getAllocated(index, 0) > 0** check
- A user withdraws **100** from Vault, so **activeStake = 100** while both group sizes remain unchanged
- Now **UniversalDelegator.getAllocated(group2, 0) = 0**, so **UniversalDelegator.removeSlot(group2)** becomes allowed
- Later, someone deposits **100** back and **activeStake = 200**. The **getAllocated(group2, 0)** will return previous saved size. It creates phantom allocation.

This may lead to unexpected behavior. For example, **UniversalSlasher.requestSlash(subnetwork, operator, amount, hints)** can be called for an operator whose group was removed. Additionally, **UniversalDelegator.stakeFor()** and related allocation getters may return values that are inconsistent.

Recommendation

We recommend zeroing the removed slot's size and clean up descedant links and key mappings (**_networkToSlot**, **_slotToNetwork**, **_operatorToSlot**, **_slotToOperator**).

MEDIUM-06	UniversalDelegator.setSize() increase during pending window inflates UniversalDelegator.getAllocated() beyond original allocation	Fixed at: ecac70d
-----------	--	--------------------------------------

Description

In **UniversalDelegator.setSize()**, when a slot's size is decreased, a pending amount is recorded to keep the old allocation slashable during the epoch window. However, when the size is subsequently increased within the same epoch window, the increase path does not cancel or reduce the existing pending. Since **UniversalDelegator.getAllocated()** computes **min(slotAvailable, size.latest()) + getPending(index, duration)**, the new higher size and the stale pending stack additively, causing **UniversalDelegator.getAllocated()** to return a value that exceeds the original allocation.

For example, a slot with **size = 200** decreased to **120** creates **pending = 80**, keeping **getAllocated() = 200**. If the size is then increased to **180** before the epoch expires, **getAllocated()** returns **min(available, 180) + 80 = 260**, which exceeds the original **200**.

Impact: A size decrease followed by a size increase within the same epoch window may inflate **UniversalDelegator.slashableStake()** beyond the vault's actual collateral, leading to over-reduction of delegator state on slash. The **owed** gap affects other stakers in the vault who bear the debt until future deposits cover it.

Recommendation

We recommend reducing existing pending when processing a size increase, so that **UniversalDelegator.getAllocated()** never exceeds the maximum of the old and new size.

MEDIUM-07

UniversalDelegator.removeSlot() and UniversalDelegator.resetAllocation()
unexpected behavior

Fixed at:
05c3b86

Description

UniversalDelegator.removeSlot(), UniversalDelegator._removeSlot(), and UniversalDelegator.resetAllocation() treat removal as a local pointer rewrite instead of a deletion. This issue appears through several different instances:

- 1. UniversalDelegator.resetAllocation() may collapse a network removal onto its parent when existChildren == 1, but it does not verify that the promoted parent is still active before calling UniversalDelegator._removeSlot(). That means a previously removed slot can be removed again through the reset path. Example:

```
1. removeSlot(removedGroup):

root (removedGroup not linked)

removedGroup (exists = false, existChildren = 1)
└─ orphanNetwork (exists = true)

2. When resetAllocation(orphanNetwork) runs:

index = orphanNetwork
parent(index) = removedGroup
removedGroup.existChildren == 1
=> index is promoted to removedGroup
=> _removeSlot(removedGroup) runs again
```

- 2. Because UniversalDelegator._removeSlot() decrements parent.existChildren in unchecked arithmetic and rewrites firstChild, lastChild, and sibling links from stored prevSlot and nextSlot values without revalidating them, removal can corrupt parent bookkeeping. The parent may retain stale child pointers after becoming logically empty, the stored child count may no longer match the actual number of children, and double-removal may underflow existChildren to 4294967295. Example:

```
Corrupted states observed:

Case A: stale pointers remain
parent.existChildren = 0
parent.firstChild = child2
parent.lastChild = child2

Case B: counter underflows
parent.existChildren = 4294967295
parent.firstChild = 0 or stale
parent.lastChild = 0 or stale
```

- 3. UniversalDelegator._createSlot() trusts parent.lastChild without checking that the recorded tail is still active. After a removal, a new child can therefore be appended behind an inactive tail. Example:

0.

root

- ├─ group1 (size 10)
- ├─ group2 (size 20)
- └─ removedGroup (size 0)
 - └─ orphanNetwork

1. removeSlot(removedGroup)

root

- ├─ group1 (size 10)
- └─ group2 (size 20)

removedGroup (exists = false)

- └─ orphanNetwork (exists = true)

2. createSlot(newTail, size = 50)

root

- ├─ group1 (size 10)
- ├─ group2 (size 20)
- └─ newTail (size 50)

removedGroup (exists = false)

- └─ orphanNetwork (exists = true)

3. resetAllocation(orphanNetwork), which promotes to removedGroup and deletes again

root.firstChild = group1

root.lastChild = newTail

reachable chain:

group1 -> group2 -> withdrawal buffer

disconnected live child:

newTail (exists = true, size = 50, prevSlot = group2)

4. **UniversalDelegator._syncPrevSums()**, **UniversalDelegator._getPrevSum()**: orphaned descendants may retain reportable stake, disconnected live children may retain stale **prevSum**, and aggregate root allocation may be overstated. Example:

0.

Actual root balance: 80

root

```
├─ group1 (size 10)
├─ group2 (size 20)
└─ newTail (size 50)
```

1. resetAllocation(orphanNetwork), deletes removedGroup again and disconnects newTail

reachable chain used by _getPrevSum():

root

```
├─ group1 (size 10, prevSum 0)
└─ group2 (size 20, prevSum 10)
```

disconnected: newTail (size 50, prevSum 30)

2. setSize(group2, 50)

root

```
├─ group1 (size 10, prevSum 0)
└─ group2 (size 50, prevSum 10)
```

disconnected: newTail (size 50, prevSum 30)

3. Reported allocations:

getAllocated(group1) = 10

getAllocated(group2) = 50

getAllocated(newTail) = 50

Total reported allocation = 110 > 80

Impact: removal and reset sequences may leave the **UniversalDelegator** tree in a state where detached descendants still report stake, inactive slots are removed again, parent child counters and child pointers diverge.

Recommendation

We recommend treating slot removal as a tree-consistent operation rather than a local pointer rewrite. In practice, that means rejecting removal of non-leaf slots or recursively deactivating descendants.

MEDIUM-08

UniversalDelegator.removeSlot() does not decrement **_noPluginsSize** for noPlugins group slots

Fixed at:
[f02848f](#)

Description

Line: [UniversalDelegator.sol#L687](#)
In **UniversalDelegator.removeSlot()**, when removing a depth-1 group slot that has **noPlugins = true** and **size > 0**, the **_noPluginsSize** state variable is not decremented. Compare with **UniversalDelegator.resetAllocation()** which properly decrements **_noPluginsSize**, and **UniversalDelegator.setSize()** which properly adjusts **_noPluginsSize** on size changes. The **UniversalDelegator._removeSlot()** internal function only handles linked-list cleanup and sets **exists = false** without touching **_noPluginsSize**.

State flow:

- 1. Group slot created with **noPlugins = true**, **size = 100**. **_noPluginsSize += 100**
- 2. Vault is slashed or emptied so **UniversalDelegator.getAllocated(groupIndex, 0) == 0** despite **size > 0**
- 3. Curator calls **UniversalDelegator.removeSlot(groupIndex)** – succeeds because **allocated == 0**
- 4. **_noPluginsSize** still includes 100 from the removed slot
- 5. Vault receives new deposits.

UniversalDelegator.allocatable() = UniversalDelegator.totalStake() – UniversalDelegator.getNoPluginsSize() – pluginsAllocated
is reduced by phantom 100.

Impact: Permanent inflation of **_noPluginsSize** reduces **UniversalDelegator.allocatable()** in **VaultV2** by the phantom amount. This limits plugin allocation capacity (plugins cannot allocate beyond the deflated allocatable value) and blocks creation of new noPlugins slots (the size check against allocatable fails). The inflation cannot be corrected because the removed slot is inaccessible (**exists=false**) to **UniversalDelegator.setSize()** or **UniversalDelegator.resetAllocation()**. For example, if phantom **_noPluginsSize** is 100 ETH and totalStake is 150 ETH, allocatable returns 50 ETH instead of 150 ETH – a 67% reduction.

Recommendation

We recommend in **UniversalDelegator.removeSlot()**, before calling **UniversalDelegator._removeSlot(index)**, check if the slot has **noPlugins = true** and decrement **_noPluginsSize** by the slot's current size. For example:

```
if (index.getDepth() == 1 && slots[index].noPlugins) {
    _noPluginsSize -= slots[index].size.latest();
}
```

MEDIUM-09	Legacy slash execution may keep pending allocation slashable and overstate slot stake	Fixed at: ecac70d
-----------	---	-----------------------------------

Description

Lines:

- [UniversalSlasher.sol#L224-L235](#)
- [UniversalDelegator.sol#L369](#)

UniversalDelegator.setSize() can create pending allocation for non-shared branches, and **UniversalDelegator.onSlash()** is the path that clears pending and size during slash settlement. In **UniversalSlasher.executeSlash()**, legacy requests (**request.createdAt < __migrateTimestamp**) do not call **UniversalDelegator.onSlash()** and only update legacy cumulative slash checkpoints. Because of this branch split, delegator-side **pending** and **size** are not reduced when legacy slash is executed, while vault stake is still reduced by **VaultV2.onSlash()**.

Example scenario:

- After migration, a tree of single group, network, and operator is created as G -> N -> O, with size=100 on the network and operator slots, and full utilization (as vault stake is 100).
- The operator slot is then reduced with **UniversalDelegator.setSize(O, 50)**, resulting in **size(O)=50**, **pending(O)=50**, and **childrenPending(N)=50**.
- A legacy slash of **80** is executed afterward; since legacy execution does not call **UniversalDelegator.onSlash()**, the tree accounting remains unchanged while vault stake drops from **100** to **20**.
- At this point, **getAllocated(O)** is still computed as **min(slotAvailable, size(O)) + pending(O)**: **getBalance(N)=20**, **getChildrenPending(N)=50**, **slotAvailable(O)=20.saturatingSub(50)=0**, so **getAllocated(O)=min(0, 50)+50=50**.

Impact: During the pending window, **UniversalDelegator.getAllocated()** may report stake above the vault's real remaining stake, which causes **UniversalDelegator.stakeFor()** and **UniversalDelegator.stakeAt()** to return inflated values and can make **UniversalSlasher.slashableStake()** exceed actual vault stake.

Recommendation

We recommend reconsidering the pending system.

Description

- Lines:
- [VaultV2.sol#L371](#)
 - [VaultV2.sol#L401](#)

The `VaultV2.claimBatch()` function iterates over `indexes` and calls `VaultV2.claim()` for each index. Each call to `VaultV2.claim()` triggers the `VaultV2.withDeallocatePlugins(true)` modifier, which in turn calls `deallocatePlugins()`. It is redundant to call `VaultV2.deallocatePlugins()` function for each index.

Recommendation

We recommend extracting the core claim logic into `VaultV2._claim()` function without the `VaultV2.withDeallocatePlugins()` modifier and apply this modifier to `VaultV2.claimBatch()` function.

```
function claimBatch(address recipient, uint256[] calldata indexes)
    public
    withDeallocatePlugins(true)
    nonReentrant
    returns (uint256 amount)
{
    unchecked {
        for (uint256 i; i < indexes.length; ++i) {
            amount += _claim(recipient, indexes[i]);
        }
    }
}

function claim(address recipient, uint256 index)
    public
    withDeallocatePlugins(true)
    nonReentrant
    returns (uint256 amount)
{
    return _claim(recipient, index);
}
```

Description

Lines: [VaultV2.sol#L618-L634](#)

The `VaultV2.swapPlugins()` function will revert without a custom error if a plugin address is not found in the plugins array. For better debuggability and off-chain error handling, it is better to revert a custom error.

Recommendation

We recommend adding a custom error `PluginNotFound()` and checking whether plugins were found or not.

Description

Line: [VaultV2Storage.sol#L169](#)
New variables were added to **VaultV2Storage** compared to the **VaultStorage** contract. 14 variables have been added, which occupy 13 slots. Due to this update, the gap should have been reduced by 13(number of new slots), but it was reduced by 12.
Vault.sol storage layout:

Name	Type	Slot
_activeSharesOf	mapping(address => struct Checkpoints.Trace256)	14
__gap	uint256[50]	15
__gap	uint256[10]	65

VaultV2.sol storage layout:

Name	Type	Slot
pluginAllocated	mapping(address => uint256)	27
__gap	uint256[38]	28
__gap	uint256[10]	66

Due to this mistake, the **MigratableEntity** storage gap was shifted by one slot compared to the previous storage.

Recommendation

We recommend changing the number of elements in the **VaultV2Storage.__gap** variable:

```
- uint256[38] internal __gap;  
+ uint256[37] internal __gap;
```

Description

Line: [VaultV2.sol#L363](#)
The **VaultV2.instantWithdraw()** function accepts a **recipient** parameter and transfers collateral to that address. However, the **IVaultV2.InstantWithdraw** event logs **msg.sender** as the first argument instead of recipient:

```
collateral.safeTransfer(recipient, withdrawnAssets);  
  
emit InstantWithdraw(msg.sender, withdrawnAssets, burnedShares);
```

The **IVaultV2.InstantWithdraw** event definition documents its first parameter as the account that received the collateral. When a caller specifies a **recipient** different from **msg.sender**, the event records an incorrect address, as the collateral is sent to **recipient** while the event attributes it to **msg.sender**.

Recommendation

We recommend passing **recipient** instead of **msg.sender** to the **InstantWithdraw** event to match the actual transfer destination. Additionally, consider adding a **withdrawer** parameter to align with the pattern used by other vault events such as **IVaultV2.Deposit**, **IVaultV2.Withdraw**, and **IVaultV2.Claim**, which all emit both the caller and the beneficiary:

IVaultV2.sol:

```
- event InstantWithdraw(address indexed recipient, uint256 amount, uint256 burnedShares);  
+ event InstantWithdraw(address indexed withdrawer, address indexed recipient, uint256 amount, uint256 burnedShares);
```

VaultV2.sol:

```
- emit InstantWithdraw(msg.sender, withdrawnAssets, burnedShares);  
+ emit InstantWithdraw(msg.sender, recipient, withdrawnAssets, burnedShares);
```


INFORMATIONAL-05	Unused storage variable in UniversalDelegator	Fixed at: <u>380463b</u>
Description Line: UniversalDelegator.sol#L105 The _cumulativeSlash mapping is declared in UniversalDelegator but is never read or written by any function in the contract: <pre>/// @dev Cumulative slashed amounts per slot. mapping(uint96 index => Checkpoints.Trace208 amount) internal _cumulativeSlash;</pre> The UniversalDelegator.onSlash() function handles slash accounting by adjusting size, pendingCumulative, clearedPendingCumulative, clearedChildrenPendingCumulative, and _clearedNoPluginsPendingCumulative, but never updates _cumulativeSlash. Similarly, no view function reads from it. Recommendation We recommend removing the _cumulativeSlash mapping from UniversalDelegator.		

INFORMATIONAL-06	Unused _rootSlot function in UniversalDelegator	Fixed at: <u>4f9b158</u>
Description Line: UniversalDelegator.sol#L993 The UniversalDelegator._rootSlot() function is declared but never called anywhere in the contract: <pre>/// @dev Return storage pointer to the root slot. function _rootSlot() internal view returns (SlotStorage storage) { return slots[0]; }</pre> UniversalDelegator._rootSlot() has no callers. All access to the root slot throughout the contract is done directly through the general slots[index] pattern. Recommendation We recommend removing the UniversalDelegator._rootSlot() function.		

Description

Lines:

- `VaultV2.sol#L294`
- `VaultV2.sol#L314`

In VaultV2, each withdrawal is identified by a per-claimer index equal to the length of that claimer’s withdrawal list at creation time. The index is stored in `_withdrawalsOfLength[claimer]` and used in `_withdrawalSharesOf[index][claimer]` and `_withdrawalUnlockAfter[index][claimer]` inside `VaultV2._withdraw()`. Unlike Vault (V1), where withdrawals are keyed by epoch and a claimer can derive which epoch to claim from time, in V2 the index is an opaque monotonic counter per account. The contract does not expose the newly assigned index to the caller or the claimer: `VaultV2.withdraw()` returns only `(burnedShares, mintedShares)` and `VaultV2.redeem()` returns only `(withdrawnAssets, mintedShares)`. The `Withdraw` event also omits the index. So after a user (or an integrator) calls `VaultV2.withdraw(claimer, amount)` or `VaultV2.redeem(claimer, shares)`, they have no on-chain way to know which index was assigned except by assuming it is `VaultV2.withdrawalsOfLength(claimer) - 1` at the time of the call.

Impact: Claimers and integrators cannot reliably know which index to pass to `VaultV2.claim()` or `VaultV2.claimBatch()`. This breaks UX and integration (e.g. frontends or keepers that need to claim after a delay). A griever can force a claimer into a large index space filled with dust, increasing gas and complexity to claim and making it easier to leave funds unclaimed or to mis-claim. The design regression versus V1 (epoch-based, predictable claim targets) exacerbates the issue.

Recommendation

We recommend returning the assigned withdrawal index from `VaultV2.withdraw()` and `VaultV2.redeem()` so the caller and claimer can store or display it. Optionally extend the `Withdraw` event with a `uint256 indexed index` (or a non-indexed `index` if many distinct indices are expected) so index can be recovered from logs.

INFORMATIONAL-08	Confusing naming for withdrawal count and withdraw/redeem return value	Acknowledged
------------------	--	--------------

Description

Lines:

- VaultV2.sol#L159
- VaultV2.sol#L294
- VaultV2.sol#L314

Two naming choices may mislead integrators and users.

1. **VaultV2.withdrawalsOfLength(address account)** – The name suggests “how many withdrawals the account has,” but the return value is used as the next withdrawal index (e.g. in **VaultV2._withdraw()** at L536). For migrated vaults it can return **__migrateEpoch + 2** for accounts with no post-migration withdrawals (legacy support), which is not the count of user-created withdrawals. Users may therefore believe they have more distinct withdrawals than they actually do, or misinterpret the value as a count instead of an upper bound for valid claim indices.
2. **mintedShares** in **VaultV2.withdraw()** and **VaultV2.redeem()** – Both functions return **(..., mintedShares)**. In **VaultV2.deposit()**, **mintedShares** denotes newly minted vault (active) shares. In **VaultV2.withdraw()** and **VaultV2.redeem()**, the same name is used for the withdrawal-queue shares credited to the claimer (used later in **VaultV2.withdrawalsOf()** and **VaultV2.claim()**). Reusing “mintedShares” for a different concept (withdrawal shares vs vault shares) can cause confusion and wrong assumptions in callers or frontends that treat all “minted shares” as vault shares.

Recommendation

We recommend renaming **VaultV2.withdrawalsOfLength()** to a name that reflects its role as the next withdrawal index or the number of withdrawal slots (e.g. **VaultV2.nextWithdrawalIndex(account)** or **VaultV2.withdrawalCount(account)** with NatSpec clarifying that after migration it may include legacy slots). Consider renaming the return value of **VaultV2.withdraw()** and **VaultV2.redeem()** from **mintedShares** to **mintedWithdrawalShares** (and updating the interface, events, and NatSpec) so it is clear these are shares in the withdrawal queue, not vault shares.

INFORMATIONAL-09	VaultV2.donate() distributes rewards proportionally but can be gamed via sandwich attack	Acknowledged
------------------	--	--------------

Description

Line: VaultV2.sol#L407

The **VaultV2.donate()** function distributes donated amounts proportionally between **activeStake** and **activeWithdrawals**. An attacker can sandwich a large donation by:

1. Depositing a large amount just before the donation (increasing their share of activeStake).
2. Letting the donation execute (their deposit captures a disproportionate share).
3. Initiating a withdrawal immediately after.

The key insight is that **VaultV2.deposit()** is permissionless (or requires only whitelisting) and the donation amount is public in the mempool:

```
uint256 withdrawalsDonated = amount.fullMulDiv(curActiveWithdrawals, curActiveStake + curActiveWithdrawals);
_activeStake.push(uint48(block.timestamp), amount - withdrawalsDonated + curActiveStake);
```

Impact: An attacker can extract a significant portion of donated rewards by front-running the **VaultV2.donate()** transaction with a large deposit, then withdrawing after the donation.

Recommendation

We recommend implementing a deposit cooldown. Alternatively, use a mechanism that distributes rewards over multiple blocks rather than all at once.

INFORMATIONAL-10	VaultV2 interface contains unused errors and omits declared IVaultV2.ClaimBatch emission	Fixed at: dc665ea
------------------	--	-----------------------------------

Description

- Lines:
- [IVaultV2.sol#L60](#)
 - [IVaultV2.sol#L65](#)
 - [IVaultV2.sol#L308](#)

IVaultV2 declares DuplicateDepositor and DuplicatePlugin, but there is no revert path in VaultV2 that uses either selector. IVaultV2 also declares ClaimBatch, but VaultV2.claimBatch() does not emit ClaimBatch after summing the amount.

Recommendation

We recommend aligning interface declarations with implementation behavior.

VaultV2.sol:

```
function claimBatch(address recipient, uint256[] calldata indexes) public returns (uint256 amount) {
  unchecked {
    for (uint256 i; i < indexes.length; ++i) {
      amount += claim(recipient, indexes[i]);
    }
  }
  emit ClaimBatch(msg.sender, recipient, indexes, amount);
}
```

IVaultV2.sol:

```
-error DuplicateDepositor();
-error DuplicatePlugin();
```

INFORMATIONAL-11	Gas optimization	Fixed at: 97168cf
------------------	------------------	-----------------------------------

Description

Line: [VaultV2.sol#L207](#)
The condition `if (migrateEpoch == 0 || index >= migrateEpoch)` is redundant. Since `index` is unsigned, `index >= migrateEpoch` is already true when `migrateEpoch == 0`. The explicit `migrateEpoch == 0` branch therefore has no behavioral effect and only adds an unnecessary comparison.

Recommendation

We recommend simplifying the condition.

```
- if (migrateEpoch == 0 || index >= migrateEpoch) {
+ if (index >= migrateEpoch) {
```

INFORMATIONAL-12	VaultV2 slashing can be blocked by plugin deallocation revert	Acknowledged
------------------	---	--------------

Description

Line: [VaultV2.sol#L435](#)

VaultV2.onSlash() executes through **VaultV2.withDeallocatePlugins(withPlugins)**, which calls **VaultV2.deallocatePlugins()** before slashing logic. In **VaultV2.deallocatePlugins()**, each plugin can be called via **IPluginBase(plugin).deallocate(...)**, followed by **collateral.safeTransferFrom(...)**, and any revert bubbles up because there is no isolation or fallback handling. **UniversalSlasher.executeSlash()** can pass **withPlugins = true** for plugin-enabled subnetworks, so slash execution enters this pre-deallocation path. As a result, when a plugin deallocation path reverts, slash execution may revert.

Recommendation

We recommend decoupling slash finalization from plugin deallocation success.

INFORMATIONAL-13	VaultV2 trusts plugin deallocate return value and can underflow accounting	Acknowledged
------------------	--	--------------

Description

Line: [VaultV2.sol#L684](#)

VaultV2._deallocatePlugin() trusts **IPluginBase(plugin).deallocate(amount)** and subtracts the returned value directly from **pluginAllocated[plugin]**. If a plugin returns more than currently allocated **deallocated**, **pluginAllocated[plugin] -= deallocated** reverts with arithmetic underflow.

Recommendation

We recommend validating or capping plugin-reported deallocation before accounting updates so vault-side bookkeeping cannot underflow on plugin-provided return data.

```

- deallocated = IPluginBase(plugin).deallocate(amount);
+ uint256 reported = IPluginBase(plugin).deallocate(amount);
+ uint256 allocated = pluginAllocated[plugin];
+ deallocated = Math.min(reported, Math.min(amount, allocated));
  if (deallocated > 0) {
    collateral.safeTransferFrom(plugin, address(this), deallocated);
-
-  pluginAllocated[plugin] -= deallocated;
+  pluginAllocated[plugin] = allocated - deallocated;
  unchecked {
    pluginsAllocated -= deallocated;
  }
}
```

INFORMATIONAL-14	UniversalDelegator.resetAllocation() may emit a parent index instead of the subnetwork slot index	Fixed at: d43d49f
------------------	--	--------------------------------------

Description

Lines:

- [UniversalDelegator.sol#L759-L761](#)
- [UniversalDelegator.sol#L796](#)

In [UniversalDelegator.resetAllocation\(\)](#), **index** is initialized as the subnetwork slot from **getSlotOfNetwork(subnetwork)**. When the subnetwork is the last child of its group (**existChildren == 1**), the function rewrites **index** to **index.getParentIndex()** before cleanup and emits **ResetAllocation(index, subnetwork)** using this rewritten value. As a result, the **ResetAllocation** event may report a group (parent) slot index while the **subnetwork** argument identifies a network slot that was looked up. This mixes two different slot domains in a single indexed field and can produce event records where the **index** does not represent the subnetwork slot associated with the **subnetwork**.

Recommendation

We recommend preserving the original subnetwork slot index for event emission and using a separate local variable for the internal removal target (which may collapse to the parent). If both values are operationally useful, consider extending the event to include both **subnetworkIndex** and **removedIndex** explicitly.

INFORMATIONAL-15	Gas optimization 2	Fixed at: ecac70d
------------------	--------------------	--------------------------------------

Description

Lines:

- [UniversalDelegator.sol#L575](#)
- [UniversalDelegator.sol#L576](#)
- [UniversalDelegator.sol#L584](#)

In **UniversalDelegator.setSize()** function, **index.getParentIndex()** is evaluated multiple times. Caching this value in a local variable may reduce gas usage and improve readability.

Recommendation

We recommend caching **index.getParentIndex()** value.

Description

Line: [UniversalSlasher.sol#L143](#)
In the post-migration branch of **UniversalSlasher.slashableStake()**, staleness uses a strict `<` check, so **captureTimestamp == block.timestamp - epochDuration** is still valid. **UniversalSlasher.executeSlash()** re-evaluates slashability with **request.createdAt**, so a request created at **t** can still be executable at **t + epochDuration**.

Recommendation

We recommend making the slasher staleness check inclusive at the boundary:
UniversalSlasher.slashableStake()

```
unchecked {
  if (captureTimestamp == 0 || captureTimestamp >= __migrateTimestamp) {
    if (
      captureTimestamp > 0
-      && captureTimestamp < block.timestamp.saturatingSub(IVault(vault).epochDuration())
+      && captureTimestamp <= block.timestamp.saturatingSub(IVault(vault).epochDuration())
    ) {
      ...
    }
  }
}
```

Description

Line: [UniversalDelegator.sol#L490](#)
When creating a depth-2 slot (network slot under a group), **UniversalDelegator._createSlot()** assigns **_networkToSlot[subnetworkOrOperator]** without verifying that the subnetwork's network component is registered in the **NETWORK_REGISTRY**. Any arbitrary **bytes32** value can serve as the subnetwork identifier.

```
if (parentIndex.getDepth() == 1) {
  if (_networkToSlot[subnetworkOrOperator].latest() > 0) {
    revert AlreadyAssigned();
  }
  _networkToSlot[subnetworkOrOperator].push(uint48(block.timestamp), index);
  _slotToNetwork[index] = subnetworkOrOperator;
}
```

Recommendation

We recommend adding a validation check:

```
if (!Registry(NETWORK_REGISTRY).isEntity(subnetworkOrOperator.network())) revert InvalidNetwork();
```

before creating the network slot.

Client's comments

There is **setMaxNetworkLimit()** which already checks it.

INFORMATIONAL-18	PluginRegistry lacks de-whitelist functionality allowing permanently whitelisted compromised plugins	Acknowledged
------------------	--	--------------

Description

The **PluginRegistry** contract only has a **PluginRegistry.whitelistPlugin()** function with no ability to remove a plugin from the whitelist. Once a plugin is whitelisted, it remains whitelisted forever. If a plugin is later discovered to be compromised, exploitable, or malicious, the registry owner cannot prevent new vaults from adding it.

Recommendation

We recommend adding a **PluginRegistry.removePlugin()** function that marks a plugin as de-whitelisted. Add a check in **VaultV2.setPluginLimit()** that verifies the plugin is still actively whitelisted (not just historically registered).

Client's comments

We aim to have as little control on our side as possible.

INFORMATIONAL-19	Legacy request UniversalSlasher.vetoSlash() writes completed flag to new slasher storage but not old slasher, creating inconsistent cross-contract state	Acknowledged
------------------	---	--------------

Description

Line: [UniversalSlasher.sol#L256](#)
When **UniversalSlasher.vetoSlash()** is called for a legacy slash request (**index < oldSlashRequestsLength**), the function marks it completed in the NEW slasher's storage:

```
_slashRequests[slashIndex].completed = true;
```

But the old slasher's record is never updated. The new slasher's **_slashRequests[slashIndex]** has **amount = 0** (uninitialized), and only **completed** is set to **true**.
The defense works because subsequent calls to **UniversalSlasher.slashRequests(slashIndex)** first load from new storage (**completed = true**), then enter the legacy branch (due to **amount == 0**), but retain the **completed = true** from the new storage load.
However, this creates a state inconsistency: the old slasher reports the request as pending while the new slasher reports it as completed. Any external contract or monitoring system that queries the old slasher directly would see stale data. More critically, if a future migration reads from the old slasher's state, it could attempt to re-process already-vetoed or already-executed slash requests.
Additionally, when **UniversalSlasher.executeSlash()** is called for a legacy request:

```
_slashRequests[slashIndex].completed = true;
```

But the old slasher's **slashRequests(slashIndex)** still returns **completed = false**. If the old slasher has any active integrations or callbacks that check request status, they would see an inconsistent state.

Recommendation

We recommend that after marking a legacy request as completed in the new slasher, also attempt to mark it in the old slasher if an appropriate interface exists. Alternatively, document that the old slasher's state is no longer authoritative after migration, and ensure no external contracts depend on it for correctness.

Client's comments

Will be noted in documentation.

Description

Lines: [UniversalSlasher.sol#L277-L295](#)

UniversalSlasher.setResolver() does not validate no-op updates and does not revert with **AlreadySet()** when **newResolver** is already the effective or pending resolver value. For a non-zero current resolver, each call rewrites **pendingResolverData** with a **block.timestamp + resolverSetDelay**, even when the input does not represent a meaningful change. This differs from **VetoSlasher.setResolver()**, which explicitly reverts on already-set values via **AlreadySet()**.

Recommendation

We recommend adding an **AlreadySet()** guard in **UniversalSlasher.setResolver()** for no-op updates, including cases where **newResolver** equals the current effective resolver, and where it matches the pending resolver value already scheduled.

Client's comments

Decided to move from "AlreadySet()" revert for most of the cases.

Description

Lines: [UniversalSlasher.sol#L116-L120](#)

When **UniversalSlasher.slashRequests()** reads a legacy slash request that originated in the old **VetoSlasher**, it first resolves the capture-time resolver and then, if that address is non-zero, replaces it with the resolver active at **block.timestamp - 1**

```
request.resolver = IVetoSlasher(__oldSlasher).resolverAt(request.subnetwork, request.createdAt, "");
if (request.resolver != address(0)) {
    request.resolver =
        IVetoSlasher(__oldSlasher).resolverAt(request.subnetwork, uint48(block.timestamp) - 1, "");
}
```

This differs from the original **VetoSlasher.vetoSlash()** behavior, where the capture-time resolver is the entity authorized to veto, and the current resolver is only checked as a safety guard to block vetoes if the resolver has since been removed:

```
address captureResolver =
    resolverAt(request.subnetwork, request.captureTimestamp, vetoSlashHints.captureResolverHint);
if (
    captureResolver == address(0)
    || resolverAt(request.subnetwork, Time.timestamp() - 1, vetoSlashHints.currentResolverHint)
    == address(0)
) {
    revert NoResolver();
}

if (msg.sender != captureResolver) {
    revert NotResolver();
}
```

Recommendation

We recommend aligning the legacy resolver lookup with the original **VetoSlasher** contract.

```
request.resolver = IVetoSlasher(__oldSlasher).resolverAt(request.subnetwork, request.createdAt, "");
if (request.resolver != address(0)) {
-   request.resolver =
-       IVetoSlasher(__oldSlasher).resolverAt(request.subnetwork, uint48(block.timestamp) - 1, "");
+   if (IVetoSlasher(__oldSlasher).resolverAt(request.subnetwork, uint48(block.timestamp) - 1, "") == address(0)) {
+       request.resolver = address(0);
+   }
}
```


Description

Line: [UniversalSlasher.sol#L140](#)
`UniversalSlasher.slashableStake()` routes all post-migration requests with `captureTimestamp == 0 || captureTimestamp >= __migrateTimestamp` to `UniversalDelegator.stakeFor(subnetwork, operator, 0)`. In this branch, the function only rejects timestamps older than `block.timestamp - epochDuration`, but it does not reject future timestamps, so a future `captureTimestamp` can still produce a non-zero slashable amount. The same branch also ignores the supplied capture time when computing stake, because it always queries the current rolling window through `UniversalDelegator.stakeFor(..., 0)` rather than stake at the requested timestamp. As a result, two different post-migration `captureTimestamp` values inside the allowed window can return the same slashable amount and include collateral that was deposited after the earlier capture time. This differs from the legacy branch of `UniversalSlasher.slashableStake()`, which rejects future timestamps and uses `UniversalDelegator.stakeAt(..., captureTimestamp, "")`, preserving capture-time-specific accounting.
Impact: Post-migration slashes are calculated from stake that did not exist at the claimed `captureTimestamp`, if the deposit happened after `captureTimestamp`. This allows slash requests to reach collateral deposited after the alleged fault time and also permits non-zero slashability for future timestamps.

Recommendation

We recommend aligning post-migration behavior with capture-time semantics by rejecting `captureTimestamp >= block.timestamp` and computing slashable stake from the stake that existed at `captureTimestamp` rather than from the current rolling window. If the intended design is to slash against the present window after migration, we recommend documenting that behavior explicitly and constraining the accepted timestamp range so callers cannot supply misleading future or stale capture times.

Description

Lines:

- [UniversalSlasher.sol#L143](#)
- [UniversalSlasher.sol#L152](#)
- [UniversalSlasher.sol#L225](#)
- [UniversalSlasher.sol#L242](#)
- [UniversalSlasher.sol#L325](#)
- [UniversalSlasher.sol#L363](#)
- [UniversalSlasher.sol#L238](#)
- [UniversalSlasher.sol#L304](#)
- [UniversalSlasher.sol#L147](#)
- [UniversalSlasher.sol#L158](#)

UniversalSlasher mixes **IVault()**, **IVaultV2()**, **VaultV2()**, and **UniversalDelegator()** casts when interacting with a vault that is explicitly required to be V2. The **IVault()** getter calls currently work only because **IVaultStorage** and **IVaultV2Storage** expose overlapping selectors such as **burner()**, **delegator()**, **slasher()**, and **epochDuration()**, so these reads can be consistently expressed through **IVaultV2()** instead of alternating between legacy and V2 types. At the same time, the need to call **VaultV2(vault).onSlash()**, **VaultV2(vault).syncOwedSlash()**, and **UniversalDelegator(...).onSlash()** shows that the V2-facing interfaces are incomplete for the slasher flow, because those methods are relied on by **UniversalSlasher** but are not declared in **IVaultV2** and **IUniversalDelegator**.

Recommendation

We recommend making the V2 dependency explicit in **UniversalSlasher** by replacing the legacy-style **IVault()** getter calls with **IVaultV2()** where the accessed selectors are part of the V2 storage surface, and introducing dedicated V2/internal interfaces for slasher-only hooks such as **onSlash()** and **syncOwedSlash()** instead of relying on concrete **VaultV2()** and **UniversalDelegator()** casts.

Description

Lines:

- [UniversalDelegator.sol#L118](#)
- [UniversalDelegator.sol#L533](#)
- [UniversalDelegator.sol#L575](#)
- [UniversalDelegator.sol#L633](#)
- [UniversalDelegator.sol#L657](#)

UniversalDelegator.slotExists(), **UniversalDelegator._createSlot()**, **UniversalDelegator.setSize()**, and **UniversalDelegator.swapSlots()** repeatedly recompute derived values such as **index.getChildIndex()** and **index.getParentIndex()** within the same execution path, and the **slotExists** modifier keeps a small revert branch inline instead of delegating to a shared internal helper. These patterns are functionally correct, but they add avoidable helper invocations, repeated stack work, and duplicated modifier bytecode in paths used during slot creation, resizing, and reordering.

Recommendation

We recommend caching **index.getChildIndex()** and **index.getParentIndex()** in local variables whenever each value is reused within the same function, and moving the **slotExists** check into a small internal function if bytecode size is a concern.

STATE MIND